



University
of Exeter

COMM117

Large Language Models and Application

Introduction Today

- RAG Paper– Recap
- Evaluation
- RAG Variants

Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks

Patrick Lewis^{†‡}, Ethan Perez^{*},

Aleksandra Piktus[†], Fabio Petroni[†], Vladimir Karpukhin[†], Naman Goyal[†], Heinrich Küttler[†],

Mike Lewis[†], Wen-tau Yih[†], Tim Rocktäschel^{†‡}, Sebastian Riedel^{†‡}, Douwe Kiela[†]

[†]Facebook AI Research; [‡]University College London; ^{*}New York University;
plewis@fb.com

Abstract

Large pre-trained language models have been shown to store factual knowledge in their parameters, and achieve state-of-the-art results when fine-tuned on downstream NLP tasks. However, **their ability to access and precisely manipulate knowledge is still limited**, and hence on knowledge-intensive tasks, their performance lags behind task-specific architectures. Additionally, **providing provenance for their decisions and updating their world knowledge** remain open research problems. Pre-trained models with a differentiable access mechanism to explicit non-parametric memory can overcome this issue, but have so far been only investigated for extractive downstream tasks. We explore a general-purpose fine-tuning recipe for retrieval-augmented generation (RAG) — models which combine pre-trained parametric and non-parametric memory for language generation. We introduce RAG models where the parametric memory is **a pre-trained seq2seq model** and the **non-parametric memory is a dense vector index of Wikipedia**, accessed with a pre-trained neural retriever. We compare two RAG formulations, one which conditions on the same retrieved passages across the whole generated sequence, and another which can use different passages per token. We fine-tune and evaluate our models on a wide range of knowledge-intensive NLP tasks and set the state of the art on three open domain QA tasks, outperforming parametric seq2seq models and task-specific retrieve-and-extract architectures. For language generation tasks, we find that RAG models generate more specific, diverse and factual language than a state-of-the-art parametric-only seq2seq baseline.

Motivation

- their ability to access and precisely manipulate knowledge is still limited
- providing provenance for their decisions and updating their world knowledge remain open research problems

RAG: Retrieval-Augmented Generation

“We explore a general-purpose fine-tuning recipe for retrieval-augmented generation (RAG) — models **which combine pre-trained parametric** and **non-parametric memory for language generation**. We introduce RAG models where the parametric memory is a **pre-trained seq2seq model** and the non-parametric memory is **a dense vector index of Wikipedia, accessed with a pre-trained neural retriever**.”

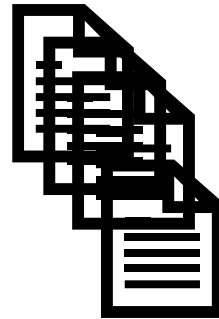
-- [*Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks*](#) , Patrick Lewis (Facebook AI Research) & Ethan Perez

e.g., Wikipedia

What is Retrieval?

Retrieval is the process of **accessing stored information** from memory and **bringing it into conscious awareness.**

- **Find relevant documents**
- **Ranking problem**
- **Similarity search**



Which of these 10,000 chunks are most relevant according to your query?

- Ranking
- Similarity search

What is Retrieval?

Lexical Retrieval (TF-IDF / **BM25**)

Word/tokens

Dense Retrieval (Embeddings)

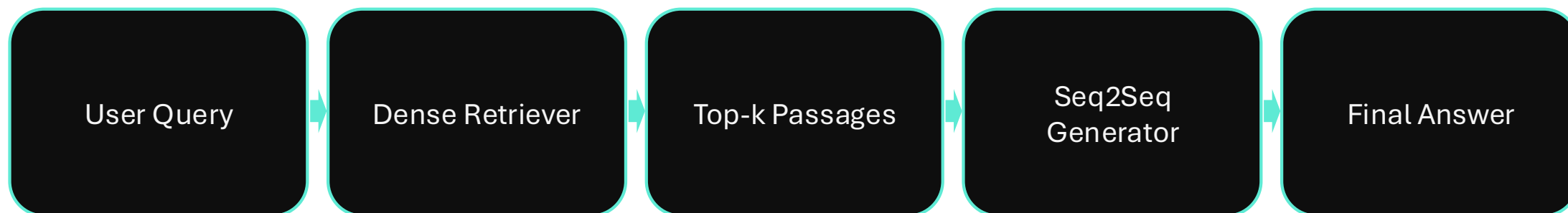
Embeddings/ **Vectors**

“Dense Retrieval is a technique that transforms **queries and documents** into **continuous, low-dimensional embeddings**, enhancing **semantic similarity** beyond simple lexical matching.”

Original Structure

downloaded snapshot of Wikipedia

- Input: question or prompt x .
- Retriever returns top- k passages z from dense index of Wikipedia.
- Generator produces output y conditioned on x and retrieved z .
- Objective: maximize $p(y | x)$ with latent retrieved documents.



Chunking in the paper

Chunking is the process of splitting large documents into **smaller pieces (chunks)** before embedding and storing them in a vector database.

- source: Wikipedia
- chunk unit: **passages**
- chunk size: **100 words**
- chunk overlap: **none stated; described as disjoint**, so effectively **no overlap**

simple fixed-size chunking strategy

Chunking

- **Fixed-size chunking**
Split every 500 **tokens**. → Simple but may cut mid-sentence.
- **Sentence-based chunking**
Group **sentences** until token limit reached. → More natural.
- **Section-based chunking**
Use document **headings**. → Best when structure exists.
- **Semantic chunking (advanced)**
Split where **topic changes** using embedding similarity.
More complex but high quality.
- **Agentic Splitting:**
Employing **LLM** to determine optimal chunking based on content and context.

“Chunking quality often matters more than model choice.”

Paper: Generator Component (BART-Style)

- **Autoregressive** sequence generation conditioned on input + passage.
- Model reads retrieved evidence as additional context.
- Output is marginalized over retrieved documents.
- Generator and retriever can be trained end-to-end.

Two Variants in the Paper

RAG-Sequence

- Retrieve once at the beginning
- Keep the same top-k retrieved passages for the entire answer
- The whole output is generated using that fixed evidence set

RAG-Token

- Retrieval can effectively change during decoding
- When generating different tokens, the model can rely on different retrieved passages
- Different parts of the answer may be supported by different documents

RAG paper - Performance

Table 1: Open-Domain QA Test Scores. For TQA, left column uses the standard test set for Open-Domain QA, right column uses the TQA-Wiki test set. See Appendix D for further details.

Model		NQ	TQA		WQ	CT
Closed Book	T5-11B [52]	34.5	-	/50.1	37.4	-
	T5-11B+SSM [52]	36.6	-	/60.5	44.7	-
Open Book	REALM [20]	40.4	-	/ -	40.7	46.8
	DPR [26]	41.5	57.9	/ -	41.1	50.6
RAG-Token		44.1	55.2/66.1		45.5	50.0
RAG-Seq.		44.5	56.8/ 68.0		45.2	52.2

Table 2: Generation and classification Test Scores. MS-MARCO SotA is [4], FEVER-3 is [68] and FEVER-2 is [57] *Uses gold context/evidence. Best model without gold access underlined.

Model	Jeopardy		MSMARCO		FVR3	FVR2
	B-1	QB-1	R-L	B-1	Label	Acc.
SotA	-	-	49.8*	49.9*	76.8	92.2*
BART	15.1	19.7	38.2	41.6	64.0	81.1
RAG-Tok.	17.3	22.2	40.1	41.5	72.5	<u>89.5</u>
RAG-Seq.	14.7	21.4	<u>40.8</u>	<u>44.2</u>		

Exact match score

RAG Paper - Performance

Table 4: Human assessments for the Jeopardy Question Generation Task.

	Factuality	Specificity
BART better	7.1%	16.8%
RAG better	42.7%	37.4%
Both good	11.7%	11.8%
Both poor	17.7%	6.9%
No majority	20.8%	20.1%

Human Evaluation

Table 5: Ratio of distinct to total tri-grams for generation tasks.

	MSMARCO	Jeopardy QGen
Gold	89.6%	90.0%
BART	70.7%	32.4%
RAG-Token	77.8%	46.8%
RAG-Seq.	83.5%	53.8%

Diversity

From Paper 2020 to Modern RAG Systems

- **Reranking** improve precision before generation.

Stage 1: Fast Retriever

Select Top-50 candidate chunks

User query:

“Can I refund a digital product?”

Retriever returns Top-50 candidates:

Chunk 17 – Refund policy section

Chunk 83 – Digital goods clause

Chunk 12 – Shipping policy

Chunk 204 – Account FAQ

...

(50 chunks total)

Stage 2: **Cross-Encoder Re-ranker**

Reordered Top-5

LLM

A **cross-encoder re-ranker** is usually:

A **Transformer encoder model (often BERT-style)** fine-tuned for relevance scoring. → Encoder only Transformer

A classification/scoring model

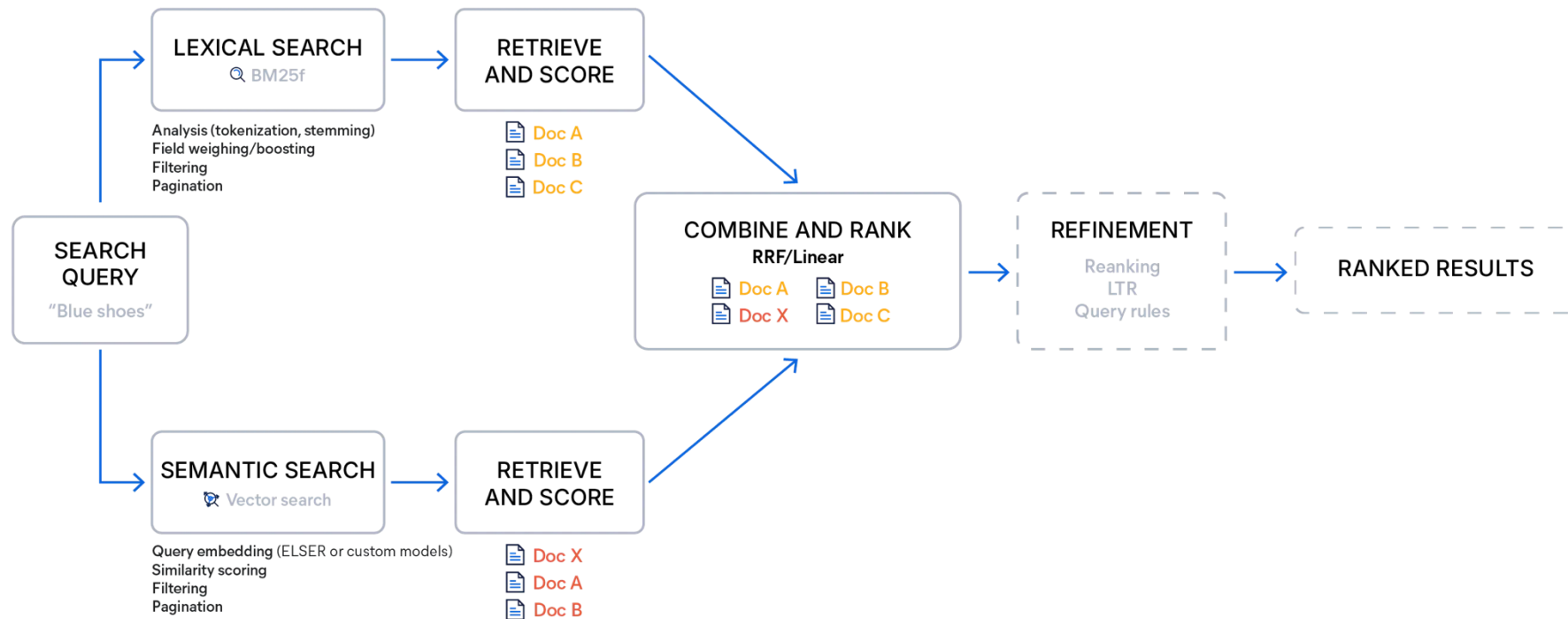
- BERT
- RoBERTa
- DeBERTa
- MiniLM

scores

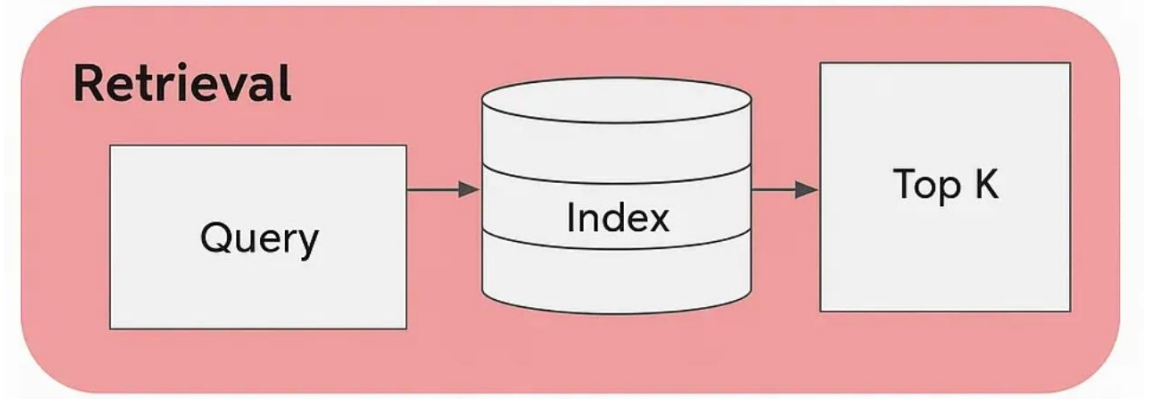
Ranking Top-5 selected

From Paper 2020 to Modern RAG Systems

- **Reranking** improve precision before generation.
- Modern stacks add **hybrid search** (dense + sparse).



Morden Retriever setting



- **Top-k**: start **5** (raise to **10** if misses; lower if noisy/costly)
- **Threshold (similarity)**: start **0.78** (raise 0.82 = cleaner; lower 0.74 = fewer empty retrieves)
- **Filters**: where to focus? (**region/version/department**) to avoid wrong doc → department="Finance" AND region="UK" AND version_date >= 2025-01-01
- **Hybrid**: **0.7 vector / 0.3 keyword** (shift toward keyword for IDs/codes)

Design Choices You Control

Retrieval Side

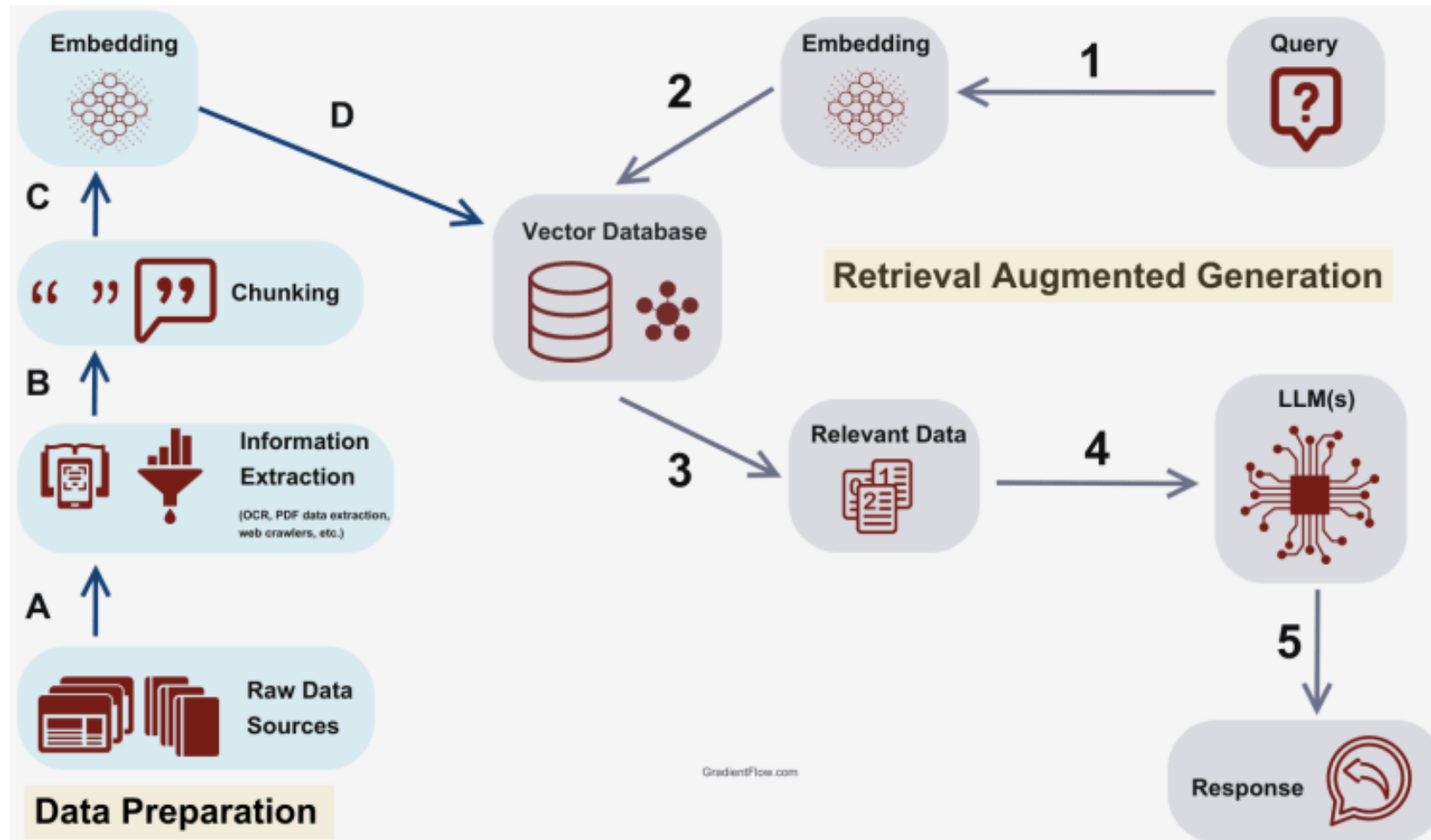
- Chunk size and overlap.
- Embedding model and index type.
- Top-k and similarity threshold.
- Reranking strategy.

Generation Side

- Prompt template and citation style.
- Context window allocation.
- Decoding settings (temperature, max tokens).
- Fallback behavior when evidence is weak.

Modern RAG- pipeline

Query → Embed → Retrieve → Prompt Template → LLM → Answer



Evaluation



If a RAG system gives a wrong answer, which component would you blame first?

- chunking
- retrieval
- reranking
- prompting
- generation

Evaluation



If a RAG system gives a wrong answer, which component would you blame first?

All possible!

Hard to evaluation

- Chunking?
- Retrieval?
- Reranking?
- Prompting?
- Generation?

Why evaluation is hard

- Multiple interacting components
 - End-to-end score is not enough
 - Good answer may hide bad retrieval
 - Bad answer may come from good retrieval
- Need retrieval-level, generation-level, system-level evaluation

From Paper 2020 to Modern RAG Systems

- **Reranking** improve precision before generation.
- Modern stacks add **hybrid search** (dense + sparse).
- **Observability and evaluation** are now first-class concerns.

Three levels of RAG evaluation

- Retrieval level: Retrieval quality
- Generation level: Answer correctness Faithfulness
- System level: Latency, Cost

Retrieval-level evaluation

- Was relevant evidence found?
- Was it ranked high enough?
- Was it precise or noisy?

Retrieval-level evaluation

- Was relevant evidence found?

Hit Rate

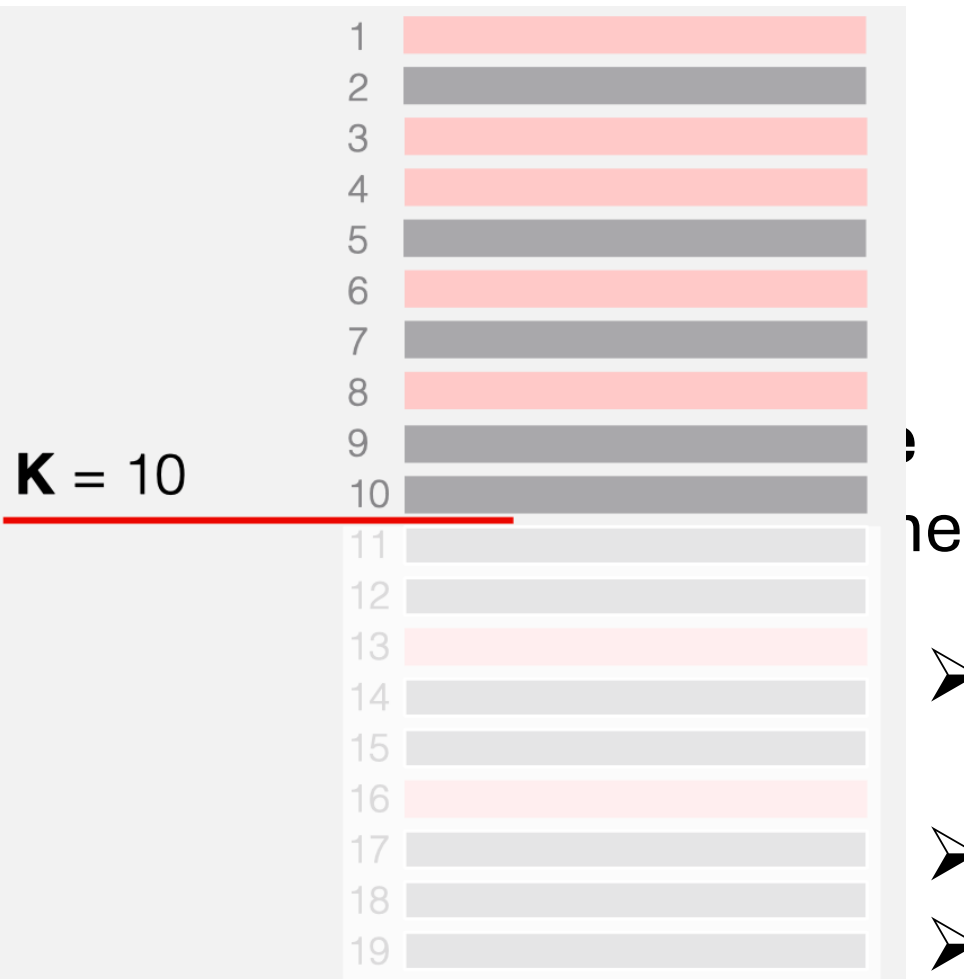
- Checks whether **at least one relevant chunk** appears in the retrieved set
- very simple yes/no view

Top-5 retrieved passages

if 1 of them is relevant $\rightarrow$ hit = 1 ; if none are relevant $\rightarrow$ hit = 0

Retrieval-level evaluation

- Was relevant evidence found?



Recall@k

- How many of the relevant chunks are found in the top-k results
- Stronger than hit rate if there are multiple useful passages

- suppose there are 4 relevant chunks in the collection
- top-5 retrieval returns 2 of them
- $\text{Recall}@5 = 2/4 = 0.5$

Was it ranked high enough?

Even if relevant evidence was found, did it appear early enough in the ranking to be useful?

MRR (Mean Reciprocal Rank)

→ focuses on the rank of the **first relevant result**

- Higher score means the first useful document appeared earlier

Formula intuition:

- relevant at rank 1 → score = 1
- relevant at rank 2 → score = 1/2
- relevant at rank 5 → score = 1/5
- no relevant result → score = 0

Was it ranked high enough?

Even if relevant evidence was found, did it appear early enough in the ranking to be useful?

MRR (Mean Reciprocal Rank)

→ focuses on the rank of the **first relevant result**

- **Higher score** means the **first useful document appeared earlier**

Formula intuition:

- relevant at rank 1 → score = 1
- relevant at rank 2 → score = 1/2
- relevant at rank 5 → score = 1/5
- no relevant result → score = 0

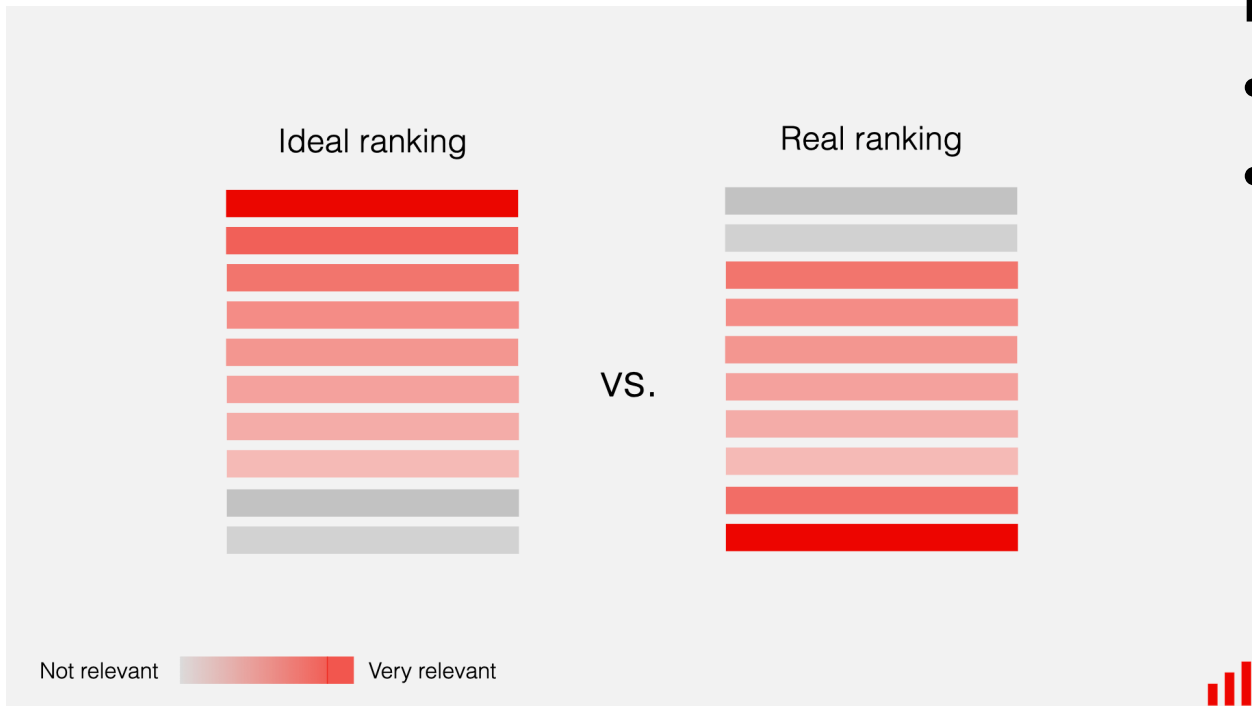
NDCG

- also evaluates ranking quality
- rewards putting more relevant items **higher up**
- useful when relevance is not just yes/no, but can be **highly relevant, partly relevant**, etc.

Was it ranked high enough?

NDCG

Discounted Cumulative Gain (DCG)



DCG

- high reward for relevant chunks
- **more reward if they appear earlier**

$$\text{DCG@K} = \sum_{k=1}^K \frac{\text{relevance at rank } i}{\text{position penalty}}$$

relevance at rank i

position penalty

$\log_2 2 = 1 ; \log_2 3 \approx 1.58 ; \log_2 4 = 2$ Larger $\rightarrow$ lower in the ranking.

Was it ranked high enough?

NDCG



Normalise

Compare this to the **ideal** ranking.

Rank	Relevance
1	3
2	2
3	1

DCG Discounted Cumulative Gain

- high reward for relevant chunks
- **more reward if they appear earlier**

$$\text{NDCG@K} = \frac{\text{DCG@K}}{\text{IDCG@K}}$$

DCG for Ideal ranking



$$\text{DCG@K} = \sum_{i=1}^K \frac{\text{relevance at rank } i}{\text{position penalty}}$$

relevance at rank i

position penalty



Was it precise or noisy?

Output Were the retrieved passages mostly useful, or did the retriever bring back lots of irrelevant clutter?

NDCG

- very useful here because it rewards rankings where the best passages are near the top
- penalizes bad ordering and too much irrelevant material high in the list

Was it precise or noisy?

Recall@k

- How many of the relevant chunks are found in the top-k results
 - 4 relevant chunks in the collection; top-5 retrieval returns 2 of them
 - $\text{Recall}@5 = 2/4 = 0.5$

Precision@k

Of the top-k retrieved chunks, how many were actually relevant?

$$\text{Precision}@k = \frac{\# \text{ relevant chunks in top-}k}{k}$$

Example

Top-5 retrieved chunks:

- 3 relevant
- 2 irrelevant

Then:

$$\text{Precision}@5 = \frac{3}{5} = 0.6$$

Was it precise or noisy?

F1@k combines Precision@k and Recall@k.

It asks:

Is retrieval both clean and complete?

Formula

$$F1@k = \frac{2 \cdot Precision@k \cdot Recall@k}{Precision@k + Recall@k}$$

Example

If:

- Precision@5 = 0.6
- Recall@5 = 0.75

Then:

$$F1@5 = \frac{2 \cdot 0.6 \cdot 0.75}{0.6 + 0.75} = \frac{0.9}{1.35} \approx 0.67$$

Retrieval-level evaluation

Recall@k

Of **all relevant chunks**, how many were retrieved **in the top-k**?
measures **coverage**

Precision@k

Of the **top-k retrieved chunks**, how many **are relevant**?
measures **noise vs usefulness**

F1@k

balances precision and recall
useful when we want both **clean and complete retrieval**

Generation-level evaluation

Did the model use the evidence correctly?

- Correctness: final answer is true
- Faithfulness : the answer is supported by the retrieved evidence
- Relevance: answer the user's question
- Completeness: include the needed information

Correct but unfaithful? Faithful but incomplete?

	Faithful	Unfaithful
Correct	ideal RAG answer	true answer, but not supported by retrieved evidence
Incorrect	grounded but wrong / incomplete interpretation	hallucination

Need to make sure everything is good

Generation-level evaluation

1. Exact Match (EM)/ F1 score

Checks whether the generated answer exactly matches the gold answer./Measures token overlap between generated answer and reference answer.

strict. → Good for short factual QA.

2. ROUGE/ BLEU

Summarisation/ translation

3 . BERTScore / semantic similarity

Compares semantic similarity, not just exact wording.

4. Human Evaluation/ QA/ LLM as judge

Faithfulness check

System – level evaluation

Did the model use the evidence correctly?

- Latency : Average response time
- Cost: Average token cost, Cost per query
- **Robustness**: answer accuracy on noisy queries
- **Reliability**: the same question asked several times

Evaluation metrics recap

Retrieval level

Hit Rate → was any relevant evidence found?

Recall@k → how much relevant evidence was found?

Precision@k → was retrieval clean or noisy?

F1@k → balance of coverage and precision

MRR → was the first relevant chunk ranked early?

NDCG → was the overall ranking good?

Generation level

correctness

faithfulness

answer relevance

completeness

System level

latency

cost

robustness

reliability

What makes a good RAG test set?

- real user-style questions
- coverage of important topics
- multi-hop / multi-source questions
- ambiguous or noisy queries
- edge cases where partial retrieval may fail

Synthetic test generation

Generating synthetic RAG test cases from the knowledge base

- start from a chunk in the knowledge base
- ask an LLM to generate:
 - a question
 - a good answer
- this creates grounded in-domain evaluation examples

but still needs human review for quality

Question

If a system retrieves the right chunk but still answers wrongly, what should we fix first?

Question

If a system retrieves the right chunk but still answers wrongly, what should we fix first?

prompt design, instruction, answer extraction, **reasoning**, output constraints.

Next steps

- RAG Variants
- Chaining, tool use, and agent frameworks- failure modes

Additional Material:

- [Evaluation of Retrieval-Augmented Generation: A Survey](#)
- <https://www.evidentlyai.com/llm-guide/rag-evaluation>

Thank you!

If you want to ask us any questions about the module, the university, or anything else, drop us an email.

Pro tip: write **COMM117** on the email subject line, to make sure we give it priority.



r.mu2@exeter.ac.uk

Questions?